Slip 8

Q.1) Write a C program that redirects standard output to a file output.txt. (use of dup and open systemcall). [10 Marks ]

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
int main() {
int fd;
// Open (or create) output.txt for writing
fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd < 0) {
perror("open");
exit(1);
}
// Duplicate fd to STDOUT_FILENO (1)
if (dup2(fd, STDOUT_FILENO) < 0) {
perror("dup2");
exit(1);
}
close(fd); // Close original fd
// Now printf outputs will go to output.txt
printf("This text is redirected to output.txt\n");
printf("Hello, this is a demo of stdout redirection using dup and open.\n");
return 0;
}
```

Q.2) Implement the following unix/linux command (use fork, pipe and exec system call)

ls –l | wc –l. [20 Marks]

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;


    if (pipe(pipefd) == -1)
    {
        printf("pipe failed");
```

```
        exit(0);
    }


    pid1 = fork();// Create the first child process for "ls -l"
    if (pid1 < 0)
     {
        printf("fork failed");
        exit(0);
    }

    if (pid1 == 0)
       {

        close(pipefd[0]); //close read end
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]); // Close the  write end
        execlp("ls", "ls", "-l", NULL);// Execute "ls -l"
        printf("execlp ls failed"); // If execlp fails
        exit(0);
    }

    // Create the second child process for "wc -l"
    pid2 = fork();
    if (pid2 < 0)
    {
        printf("fork failed");
        exit(0);
    }

    if (pid2 == 0)
    {

        close(pipefd[1]);//close write end


        dup2(pipefd[0], STDIN_FILENO); // Redirect stdin to the read end of
the pipe
        close(pipefd[0]); // Close the read end
        execlp("wc", "wc", "-l", NULL);// Execute "wc -l"
        printf("execlp wc failed"); // If execlp fails
        exit(0);
    }


    // Close both ends of the pipe
    close(pipefd[0]);
    close(pipefd[1]);
```

```
    // Wait for both child processes to terminate
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```